

抖音大型应用架构演进与思考

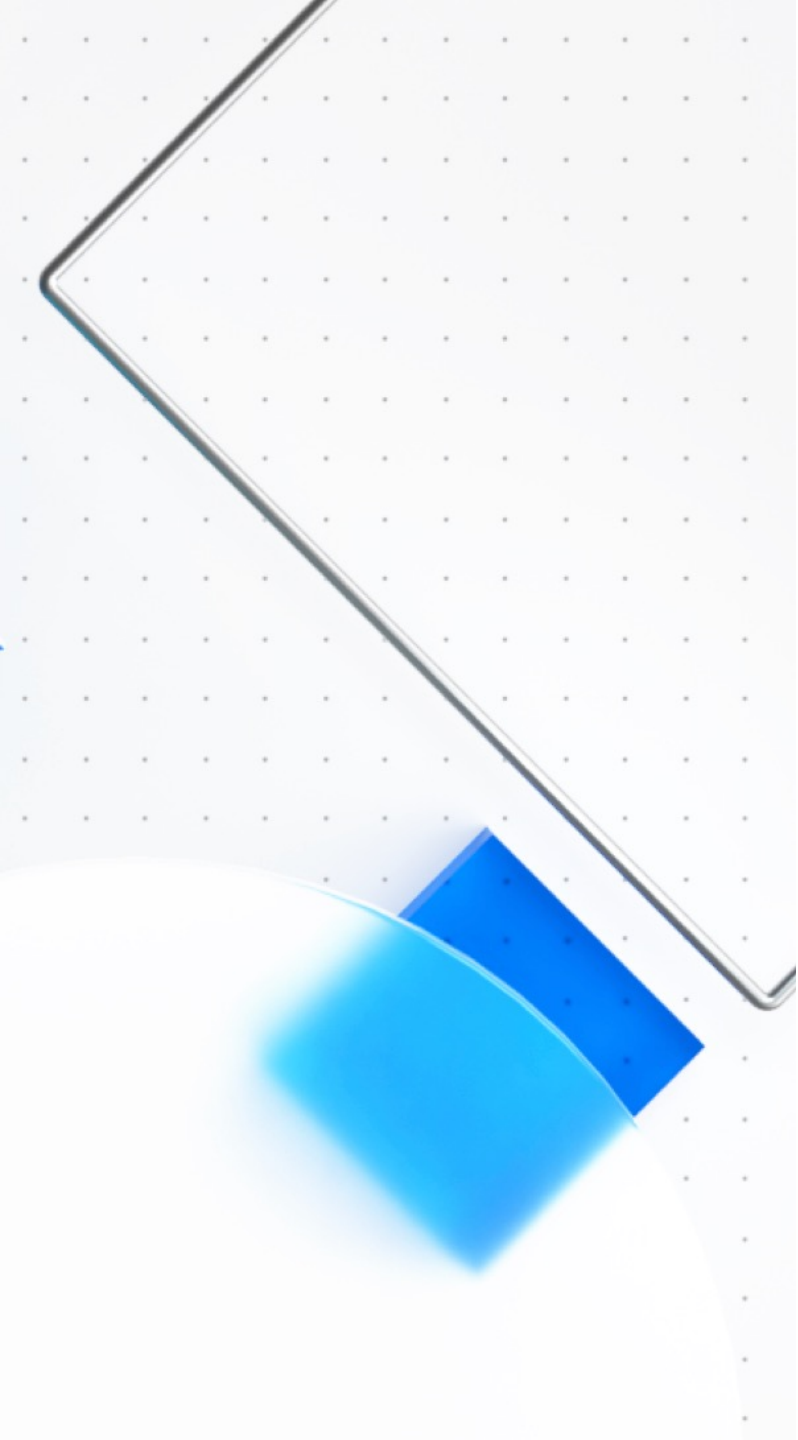
万鑫

讲师自我介绍



万鑫

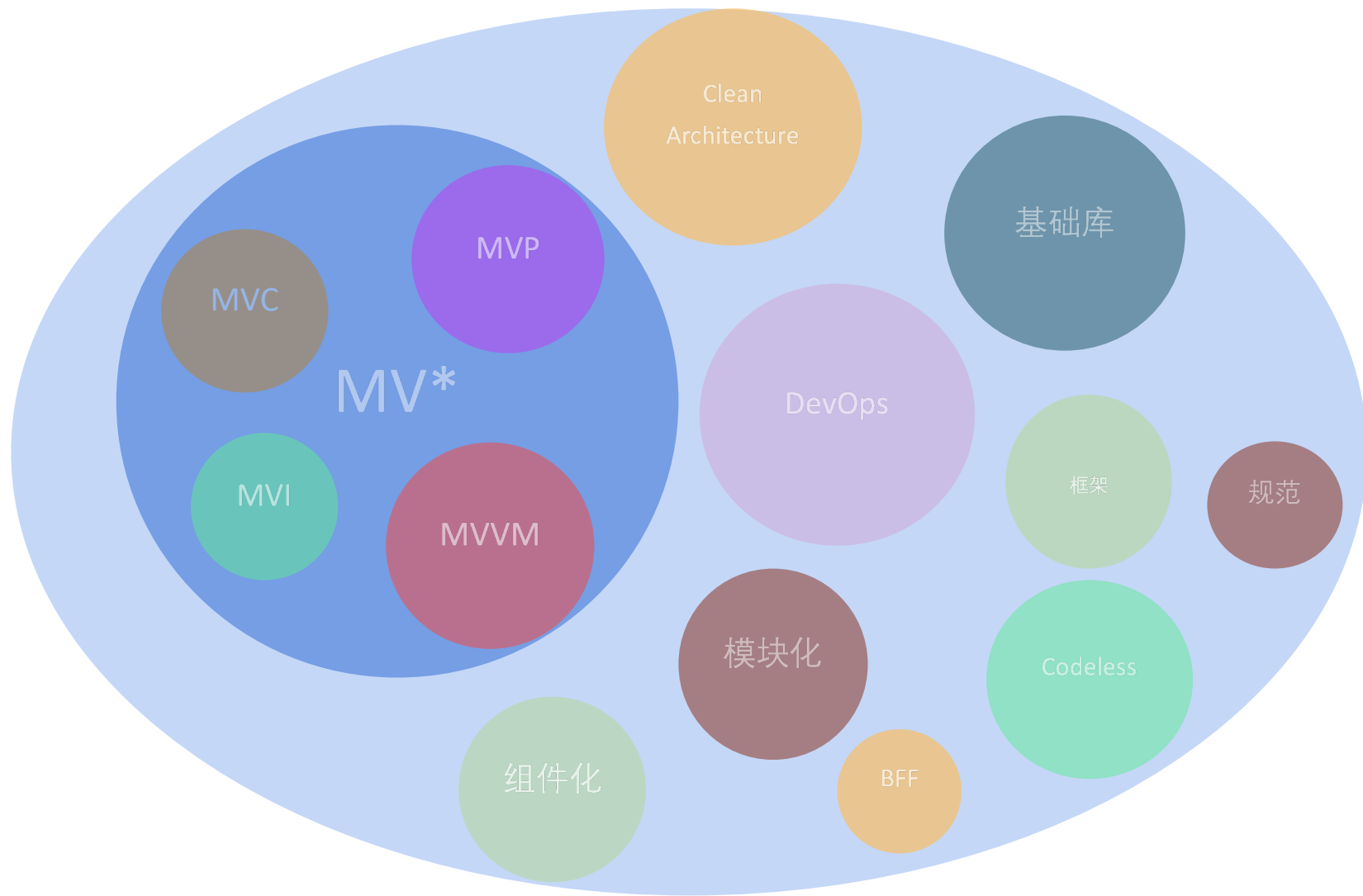
从 2009 年开始从事 Android 客户端开发，先后从事过多个大型应用的架构设计和开发工作，2020 年加入字节跳动，从事抖音工程架构及相关工具链的研发工作，对大型 App 的工程架构设计和相关优化有着较为丰富的经验。

- 
- 01 背景与目标
 - 02 架构演进路线
 - 03 抖音架构实战
 - 04 效能平台化建设
 - 05 挑战与未来展望

01

背景与目标

什么是架构



Architecture is like teenage sex,
everybody talks about it,
nobody really knows what is it.

-- 《架构漫谈》



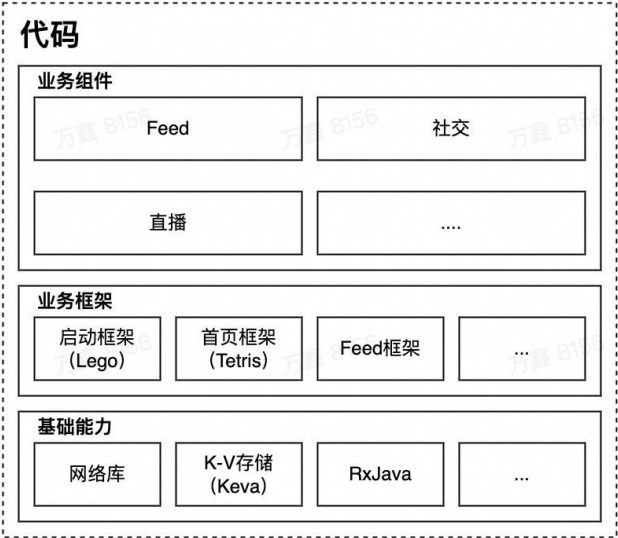
架构的目标



- 快：加快版本交付周期；如写码更快、协同更顺、编译更快
- 好：提升版本交付质量；如代码规范、代码质量；
- 省：降低研发人力成本；如代码复用；

提升研发效能

架构全景



02

架构演进路线

业务演进时间线



开始总是简单一些

2016 抖音



诉求

- 交付速度

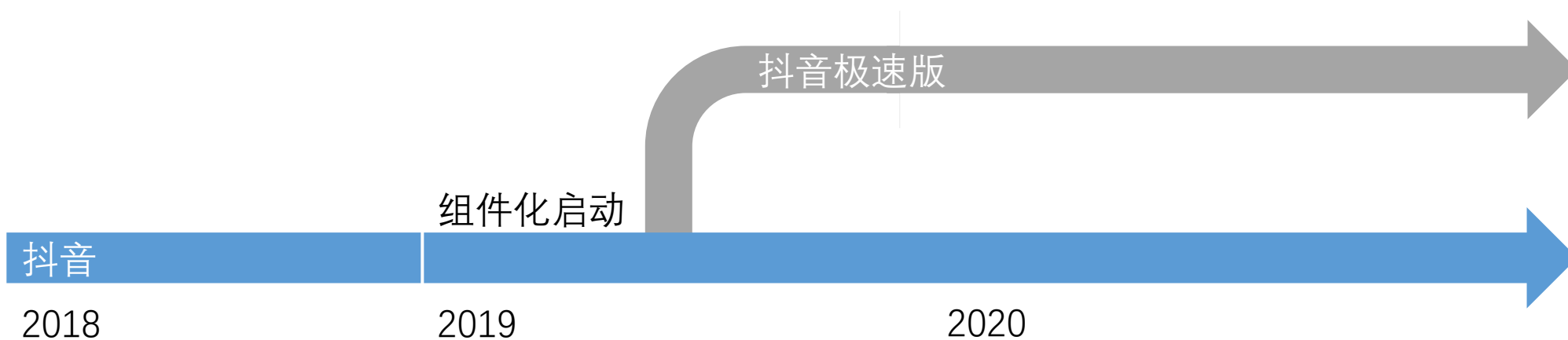
特点

- 团队规模小，代码量小
- 协同成本低

技术方案

- 单仓单组件

逐渐开始复杂



诉求

- 研发效率
- 差异迭代

特点

- 团队与代码规模中等
- 协同成本较高

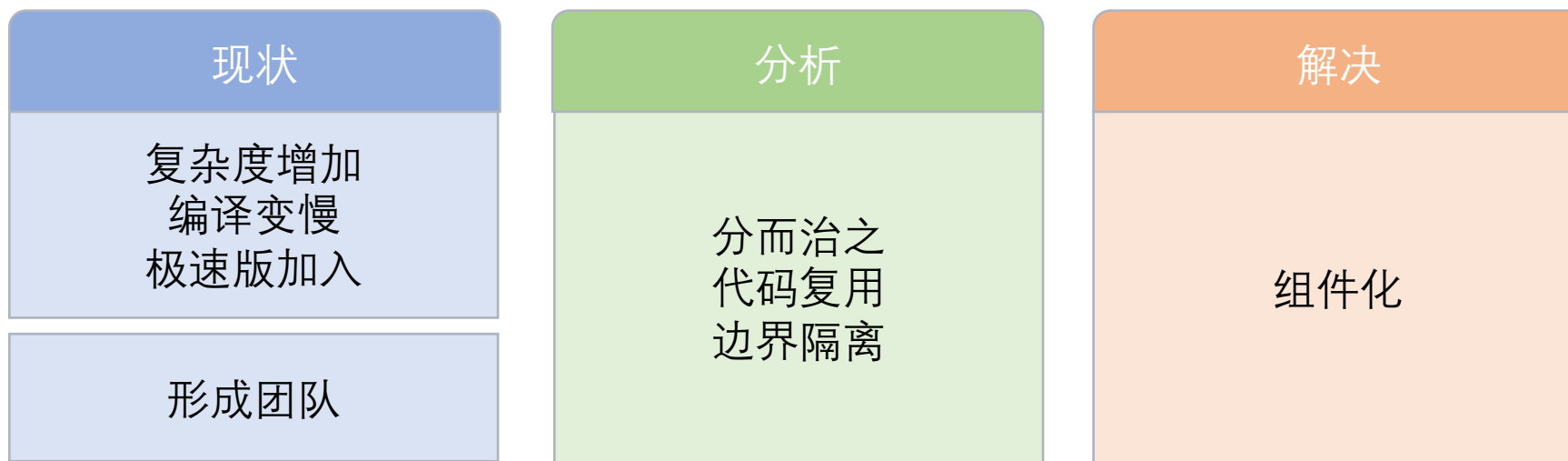
仓库隔离

- Fork仓库

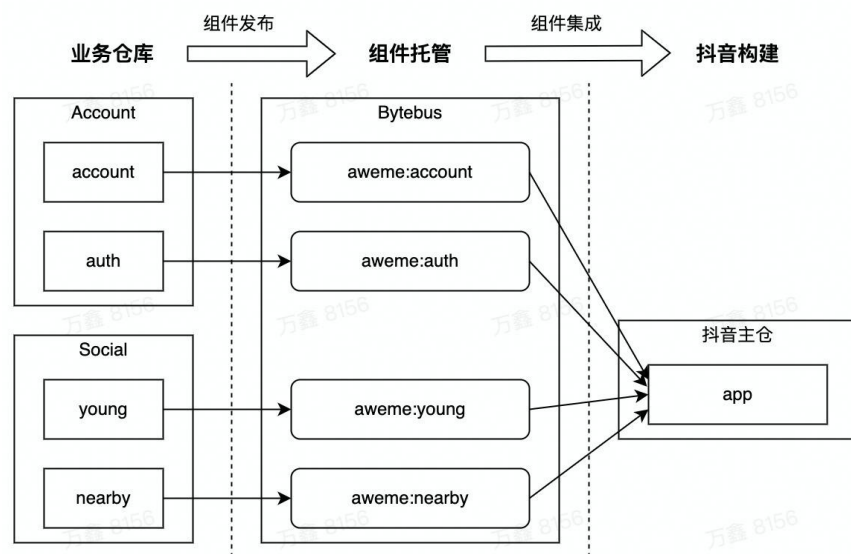
业务隔离

- 组件化

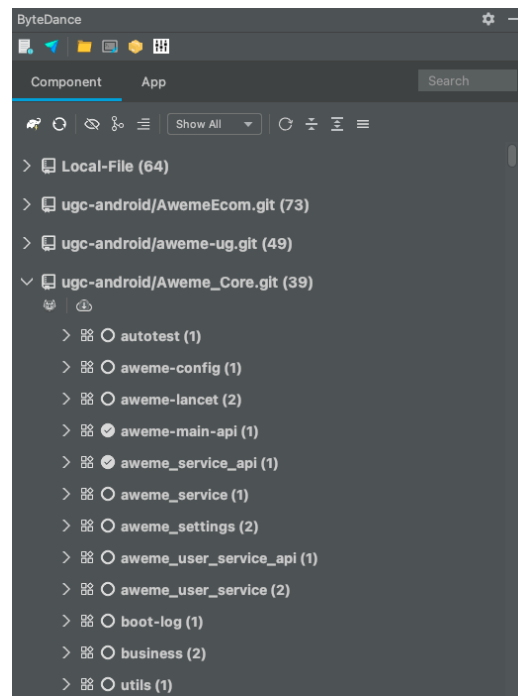
组件化



组件化



工具支持



流程保障

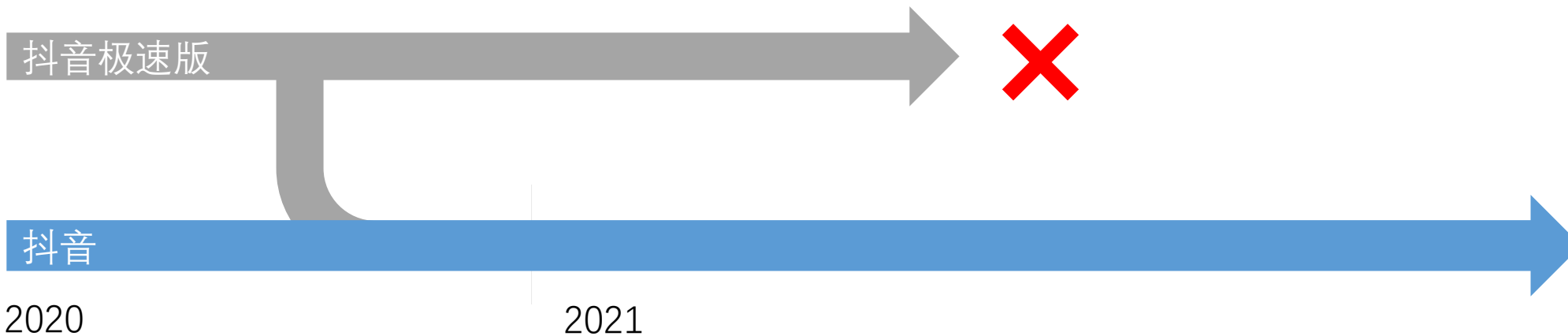


优势和劣势

- **编译速度**：彻底组件化后，本地开发时的编译速度有了较大的提升；
- **业务内聚**：业务代码聚合到业务仓库，研发只需关注少量源码即可开发，降低认知负担；
- **权限隔离**：多仓天然具备权限隔离，可以根据不同需求设置仓库的访问性，保密性更高；
- **功能复用**：组件可以在多个 App 中进行复用，降低多端开发成本；

- **构建速度**：初期由于组件化架构引入、拆分不够彻底等原因会导致构建速度不降反增；
- **上手成本**：组件化、多仓的使用都会带来额外的认知和使用成本；
- **研发流程**：多仓和组件发布的引入提升了合码的难度；

合就没那么简单了



诉求：

- 研发效率
- 功能同步

特点：

- 团队与代码规模大
- 协同成本高

架构方案

- Flavor 隔离
- 插件化

抖极融合

业务视角

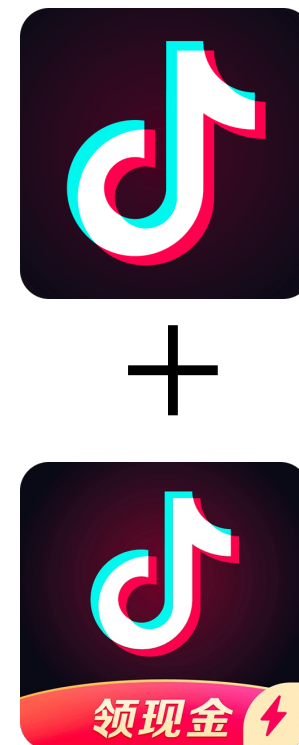
1. 抖极 = 抖音 + 差异化
2. 双端同步发布
3. 极速版 < 80M

研发视角

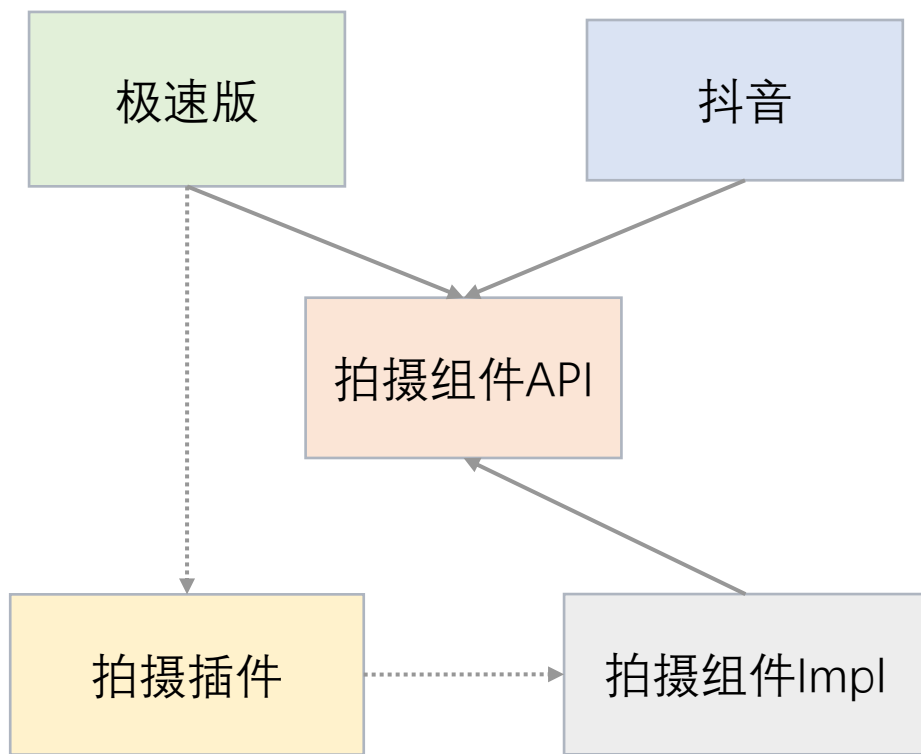
1. 90% 代码相同
2. 封板日人力集中
3. 常规优化难以达到80M

解决思路

1. 双端共基线
2. 研发流程复用
3. 使用插件化



插件框架



挑战

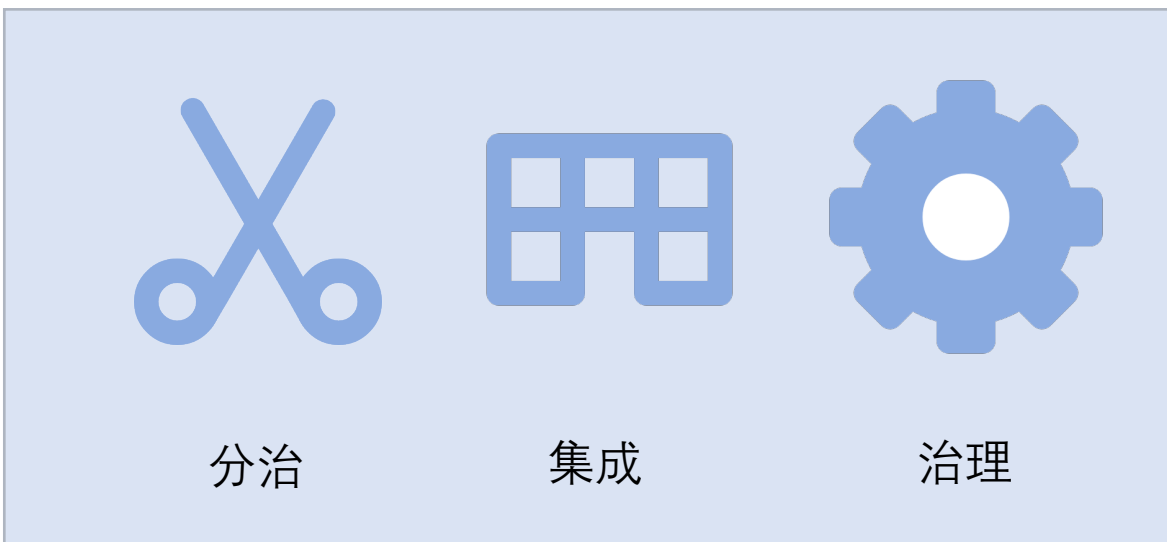
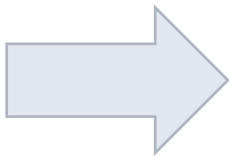
- **边界问题**：插件的物理隔离特点对组件边界的隔离性提出了更高的要求
- **管理问题**：大规模使用插件带来大量的兼容性问题，导致插件管理复杂度显著提升
- **复用问题**：针对大小包相同业务不同形态差异需要研发维护两套服务调用代码，引入较高的开发和维护成本

03

抖音架构实战

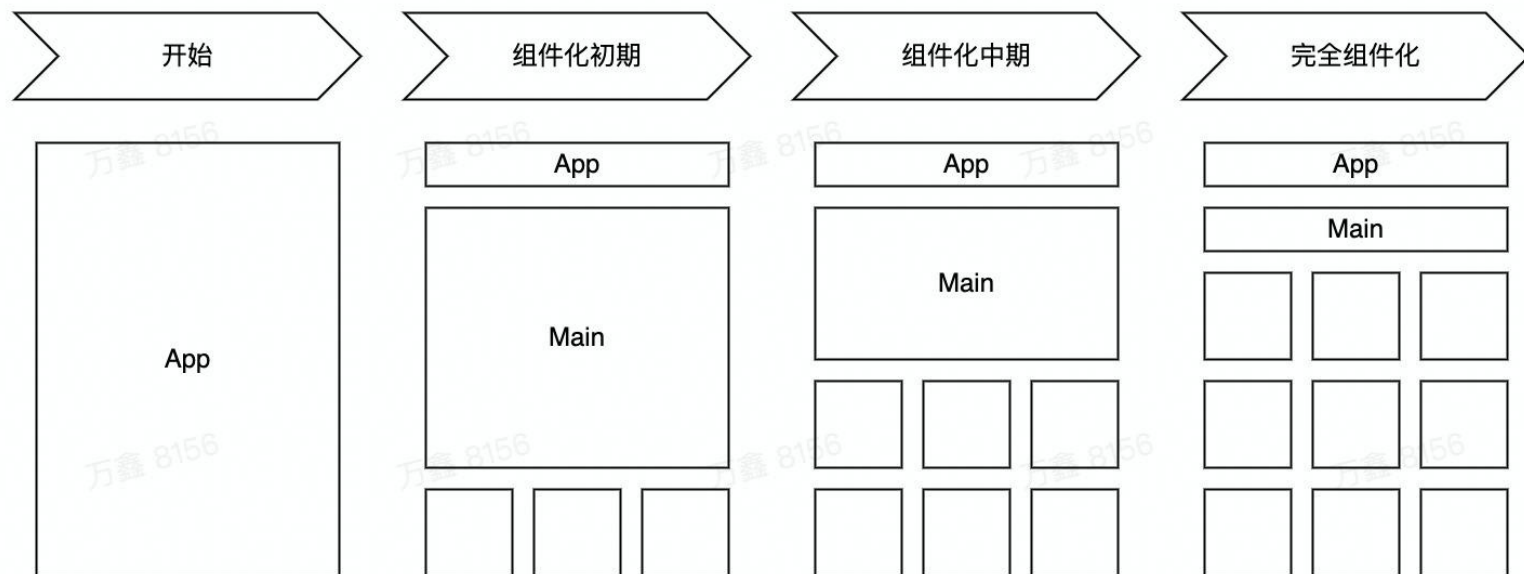
当组件化碰上超大规模

指标	增长率
总代码行	51.58%
业务组件数	69.87%
总组件数	37.00%
仓库数	31.03%





组件拆分

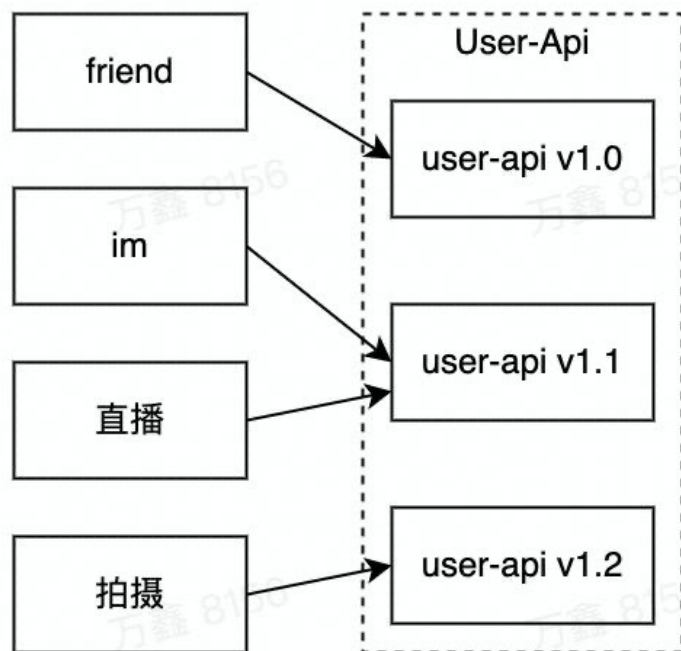


控制粒度和节奏：

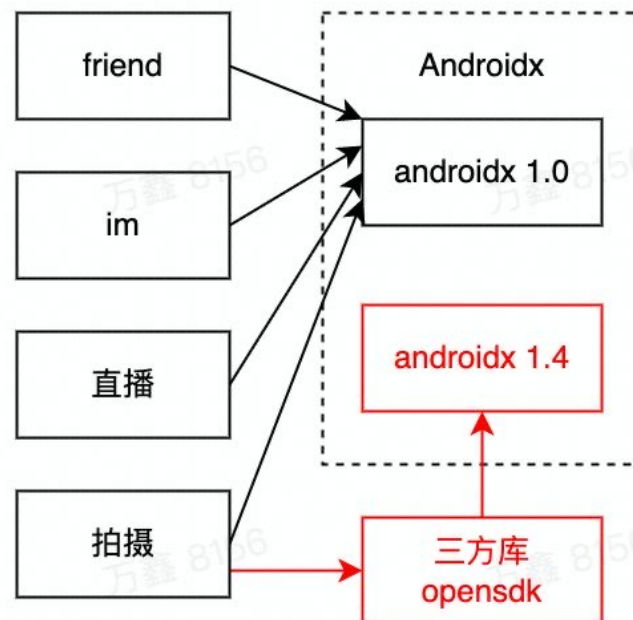
- 渐进式拆解
- 坚持目标
- 控制中心化组件

依赖管理

版本冗余



基础库不稳定

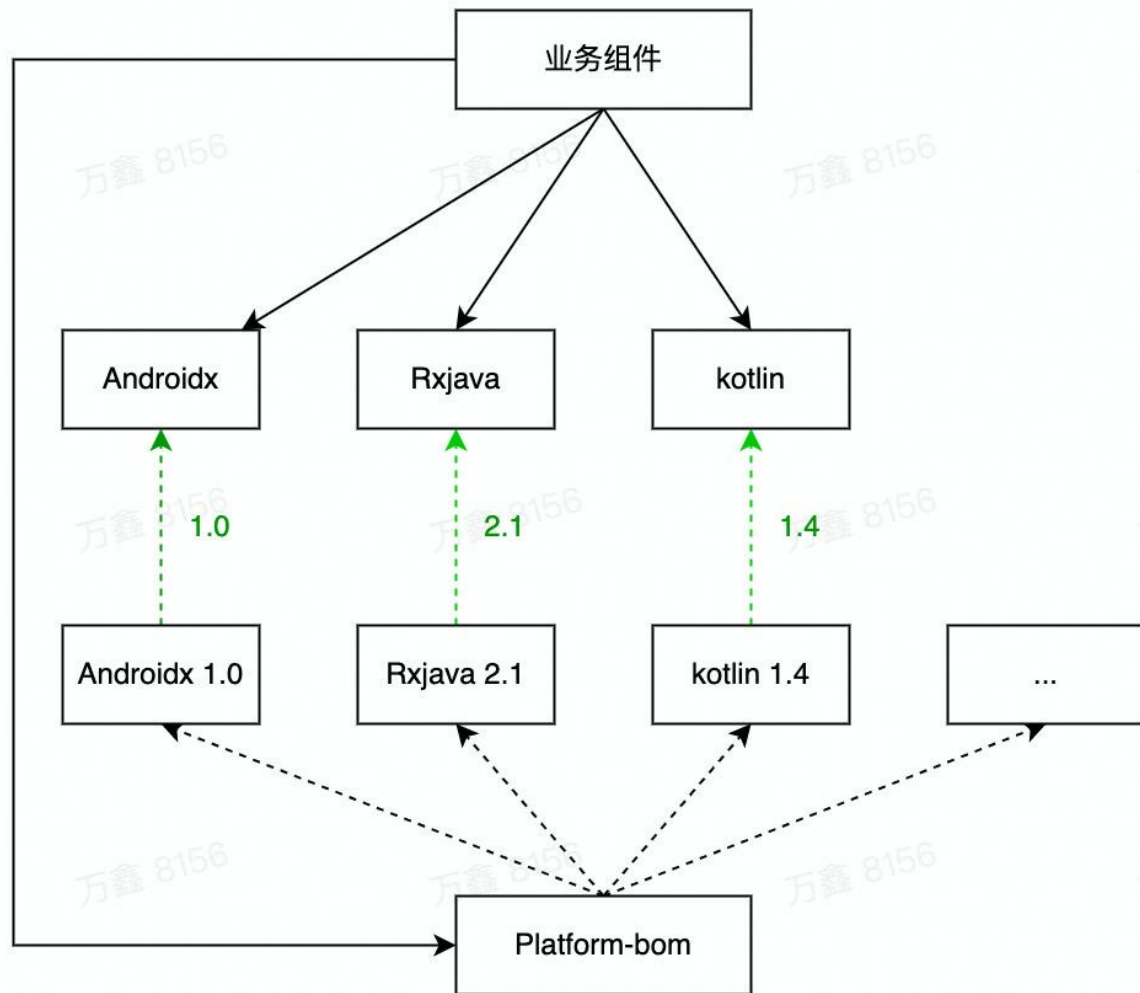


依赖管理

解决思路

- 版本管理：One Version
- 版本仲裁：锁定版本
- 依赖定义：无版本号

```
implementation("com.bytedance.ies.ugc.aweme:appcontext") ^ 15.1.0.1
implementation("com.bytedance.ies.ugc.aweme:aweme-account-api") ^ 15.3.0.1
implementation("com.bytedance.ies.ugc.aweme:aweme-network-api") ^ 15.4.0.1
implementation("com.bytedance.ies.ugc.aweme:aweme-network-lib") ^ 15.1.0.1
```



详细：
<https://docs.gradle.org/current/userguide/platforms.html#sub:using-platform-to-control-transitive-deps>

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.bytedance.ies.ugc.aweme</groupId>
  <artifactId>sdk-enforced-platform</artifactId>
  <version>23.6.1-alpha.143-test-SNAPSHOT</version>
  <packaging>pom</packaging>
  <dependencyManagement>
    ...
  </dependencyManagement>
  <description>{"sha1":"1cc73b75928e757d6ff450799c39c"}</description>
</project>
```

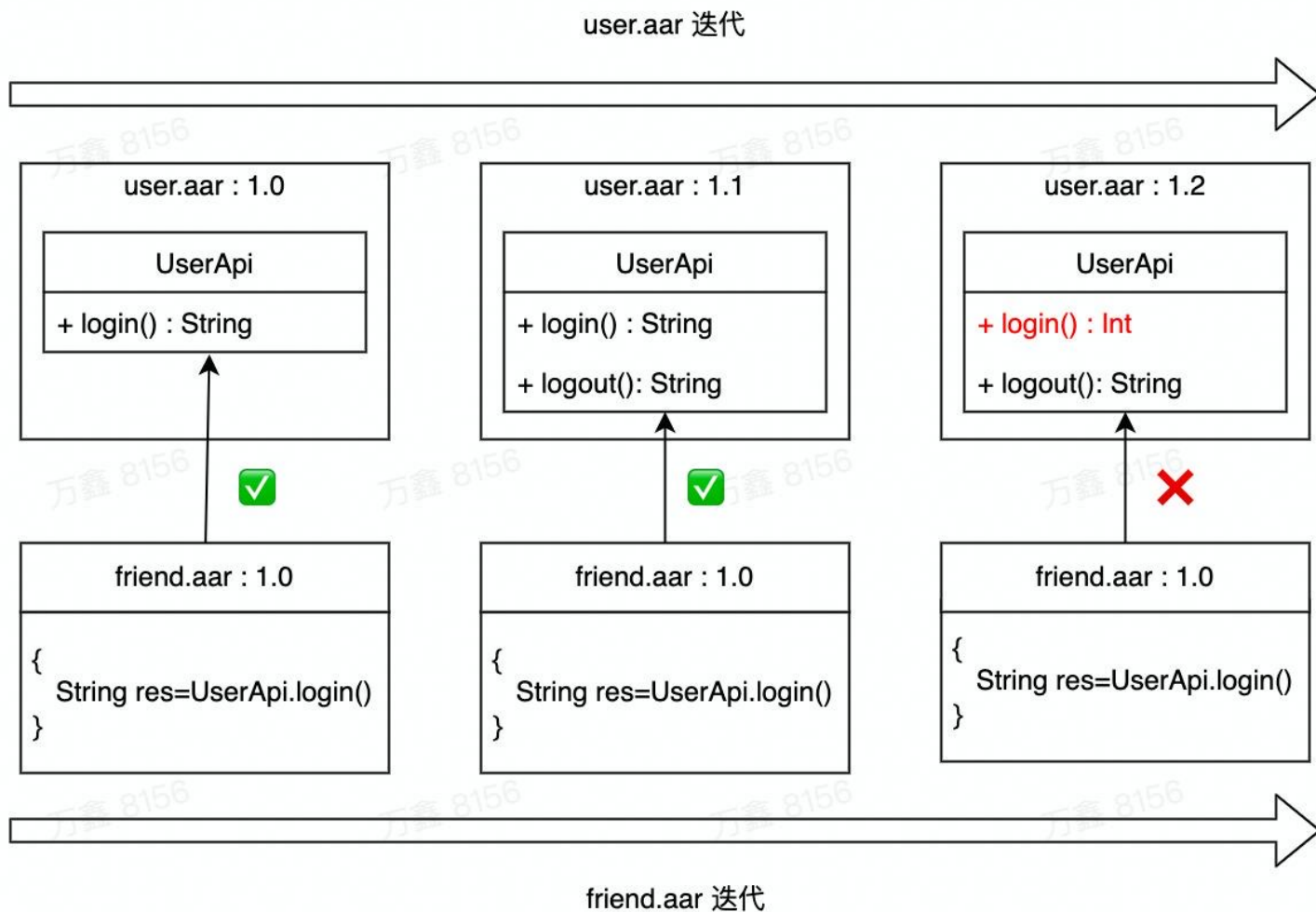
分治

集成

治理



组件兼容性



保证向前兼容？

- 冗余代码
- 包大小
- 高速迭代

二进制兼容性参考：

http://wiki.eclipse.org/Evolving_Java-based_APIs_2

字节码校验

可达性校验

类可达性
方法可达性
字段可达性

继承关系校验

继承的完整性
final 覆盖
降低访问权限
extend 接口
implement 非接口

黑名单校验

hashCode()

=====字节码错误日志开始=====

我们对字节码进行了兼容性检查，比如调用的类、函数、字段是否存在，函数原型和访问权限是否正确等，请按照下面的错误提示进行修改。

Error1

[此属性缺失]: com/ss/android/ugc/aweme/feed/model/Aweme.poiDCardStruct

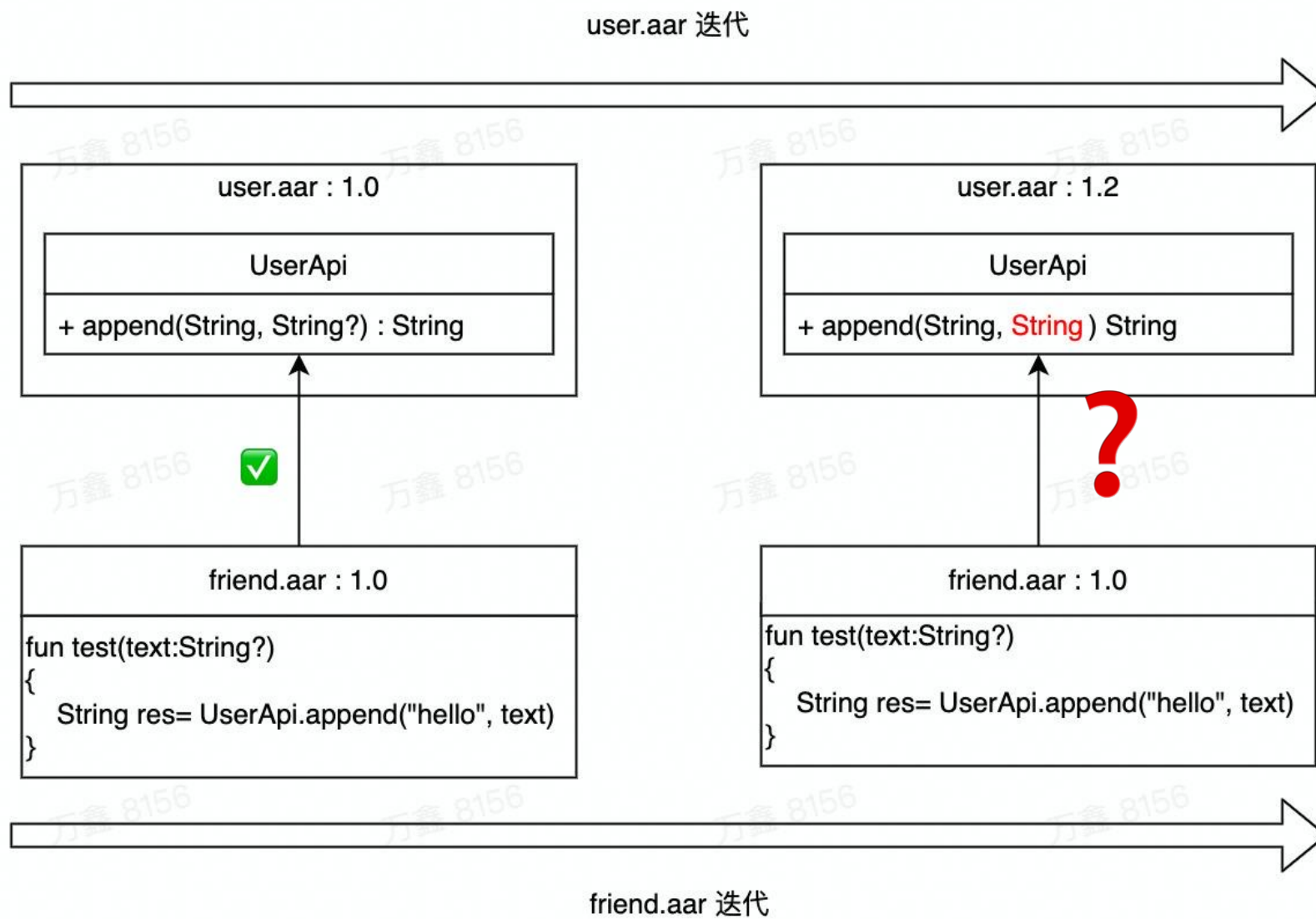
调用点位置:com/ss/android/ugc/aweme/poi/dcard/DCardConfig\$Companion.from(DCardConfig.kt:20)

非法调用: ->com.ss.android.ugc.aweme.feed.model.poi.PoiDCardStruct com.ss.android.ugc.aweme.feed.model.Aweme.poiDCardStruct

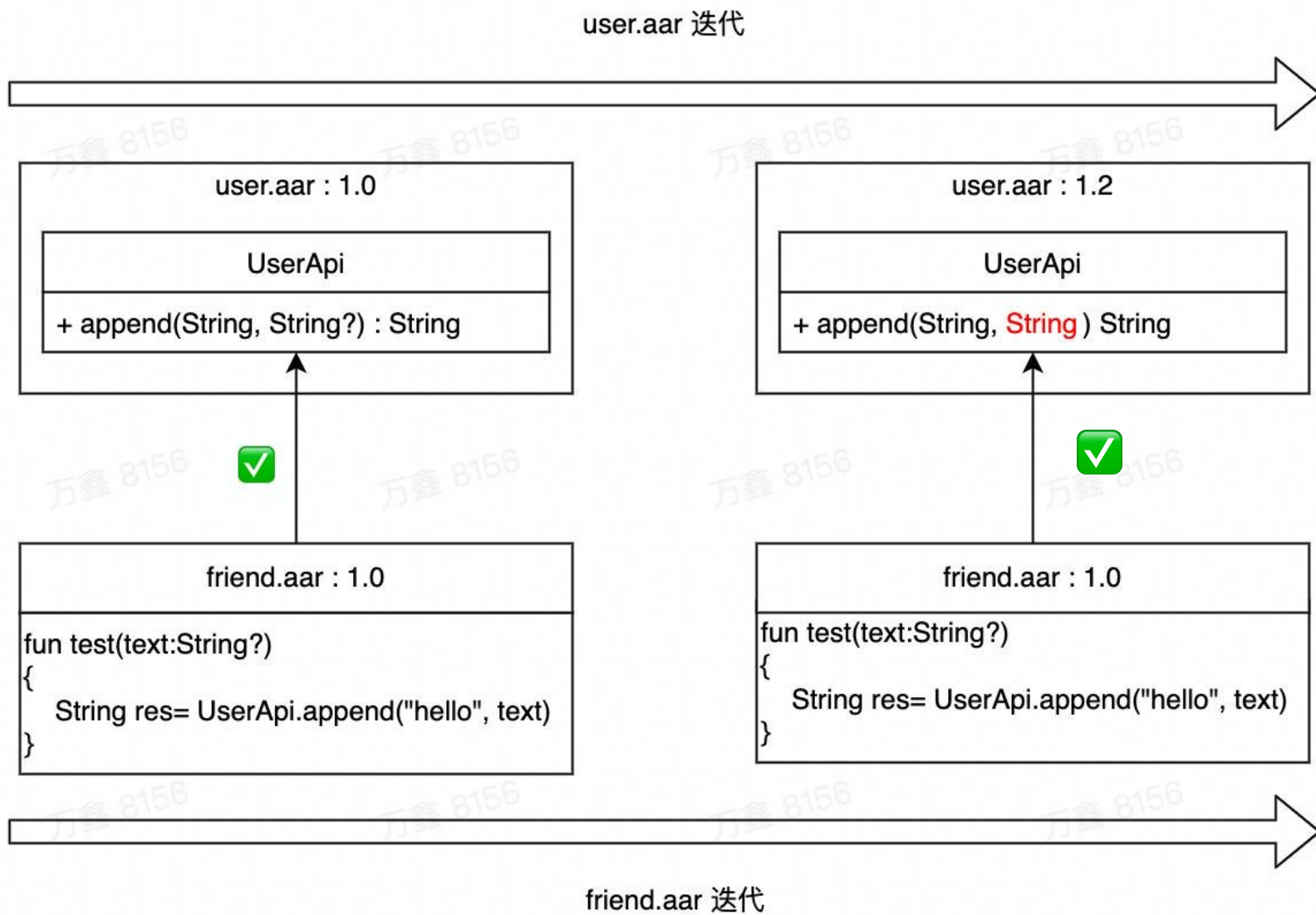
调用点所在组件:*/caches/transforms-2/files-2.1/90bfa619fa76a59a1aab2c194fa8c51f/poi-dcard-douyinCn-23.6.0.1/jars/classes.jar!com/ss/anc

开源地址：
<https://github.com/bytedance/ByteX/tree/master/reference-check-plugin>

真的没问题么?



二进制兼容，源码不兼容



```
fun test(text:String?) {
    append(a: "hello", text)
}
```

[TYPE_MISMATCH] Type mismatch.
Required: String
Found: String?

Surround with null check More actions

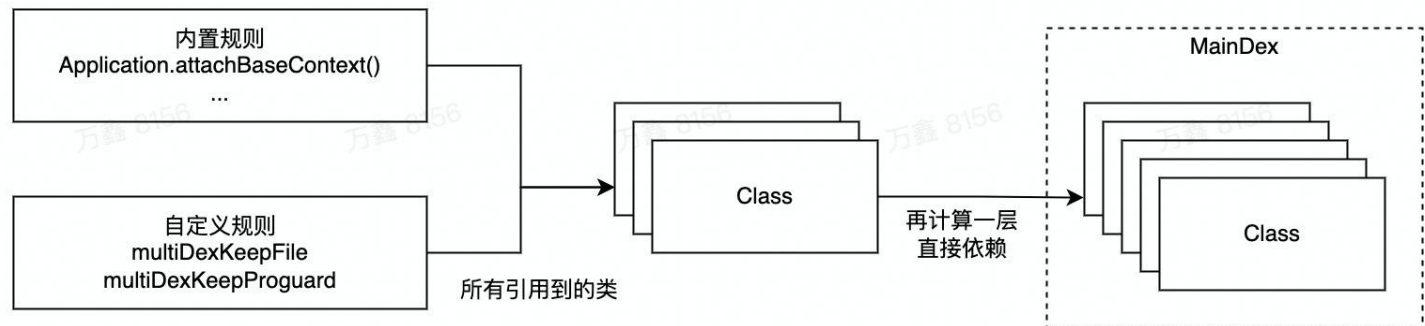
value-parameter text: String?

AwemeTools.utils

main – Dex 超限



Main dex 超限



核心的原因：

attachBaseContext方法间接
依赖了过多的类

```
class TestApplication : MultiDexApplication() {  
    override fun attachBaseContext(base: Context?) {  
        APM.start()  
        super.attachBaseContext(base)  
        GlobalXXX.init()  
    }  
}
```

解决思路

```
class TestApplication : MultiDexApplication() {  
    override fun attachBaseContext(base: Context?) {  
        APM.start()  
        // super 会调用 MultiDex.install  
        super.attachBaseContext(base)  
        GlobalXXX.init() ←  
    }  
}
```

```
class TestApplication : MultiDexApplication() {  
    override fun attachBaseContext(base: Context?) {  
        APM.start()  
        super.attachBaseContext(base)  
        afterAttach(base)  
    }  
    @MainDexIgnore  
    private fun afterAttach(base: Context?) {  
        Partition().afterAttach(base)  
    }  
    @MainDexIgnore  
    class Partition {  
        fun afterAttach(base: Context?) {  
            GlobalXXX.init()  
        }  
    }  
}
```

- 调用隔离
- 自动完成

R 文件超限

```
R.java:21511: error: code too large  public static int Ad_authorization_notification_title = 0x7f152A94;
```


R 文件是什么

- 资源是通过 ID 检索
- R文件：Name - ID 映射
- 编译时R合并累加（R.txt）
- 依赖上层R文件剧烈膨胀

```
public void setContentView(int layoutResID) {  
    // ...  
}
```

int cursor_drawable = 0x7f020000

package 资源类型 资源索引

```
int Ad_authorization_notification_title = 0x7f152A94;
```

真实的原因 – 指令数溢出

```
final int cursor_drawable = 0x7f020000
```

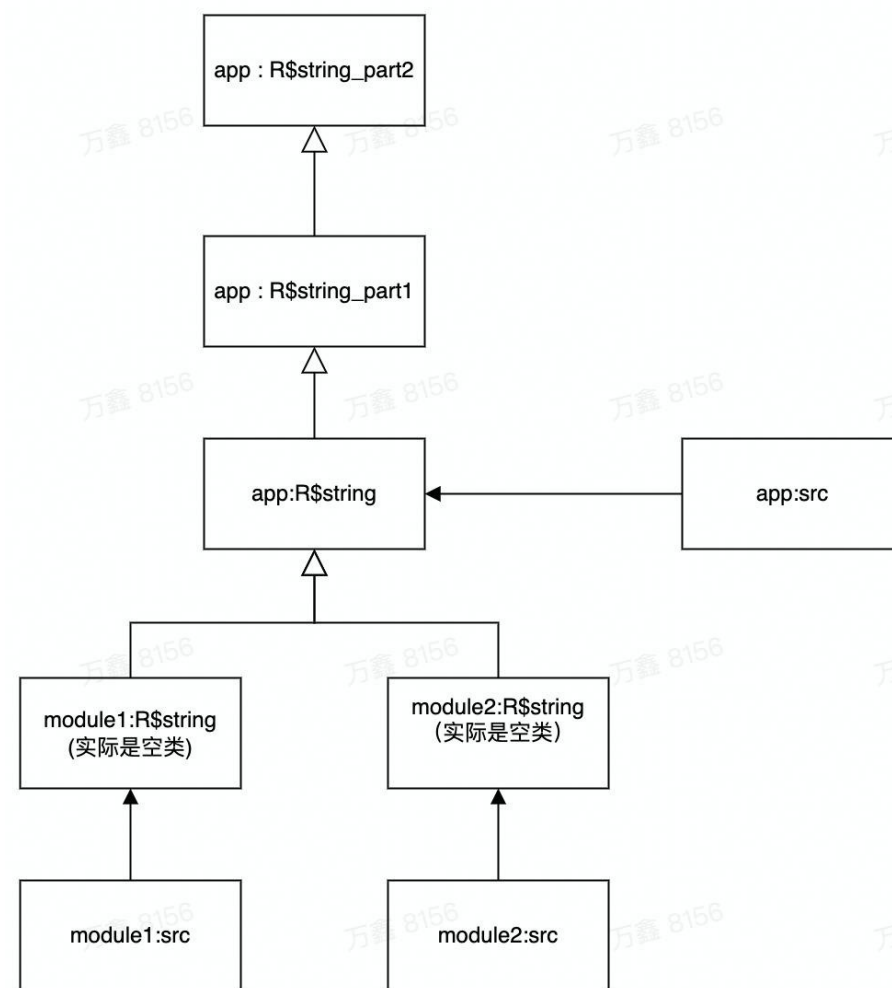
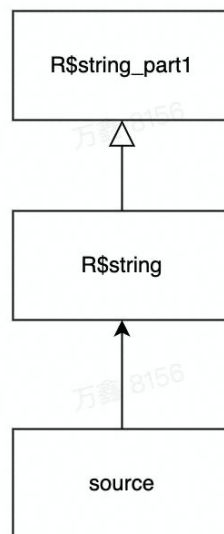
- 一个方法中指令数 < 65535
- 一个赋值语句占6个指令数
- library < 10900
- application < 32700

```
109443 65488: putstatic #21875 // Field string10935:I
109446 65489: ldc_w #21874 // int 2132093624
109447 65492: putstatic #21875 // Field string10936:I
109448 65495: ldc_w #21876 // int 2132093625
109449 65498: putstatic #21877 // Field string10937:I
109450 65501: ldc_w #21878 // int 2132093626
109451 65504: putstatic #21879 // Field string10938:I
109452 65507: ldc_w #21880 // int 2132093627
109453 65510: putstatic #21881 // Field string10939:I
109454 65513: ldc_w #21882 // int 2132093628
109455 65516: putstatic #21883 // Field string10940:I
109456 65519: ldc_w #21884 // int 2132093629
109457 65522: putstatic #21885 // Field string10941:I
109458 65525: ldc_w #21886 // int 2132093630
109459 65528: putstatic #21887 // Field string10942:I
109460 65531: return
109461 LineNumberTable:
109462   line 3: 0
109463   line 4: 5
109464   line 5: 10
109465   line 6: 15
109466   line 7: 20
109467   line 8: 25
109468   line 9: 30
109469   line 10: 35
109470   line 11: 40
109471   line 12: 45
109472   line 13: 50
109473   line 14: 55
```



R 文件超限解决思路

- 切割与继承
- 调整生成顺序





架构治理

现象	原因	治理思路
Main-Dex 超限 启动性能慢	启动路径上执行了过多的方法	控制增长 • 限制增量 • 监控变化
R 文件超限	组件的依赖深度过深	依赖树拍平 • 参考设计模式
编译速度慢	存在大型的中心化组件	剥离到业务 控制增长 • 限制文件添加/修改
	组件的依赖树过深 组件的依赖过多	依赖树拍平 依赖合理性 • 无用依赖

04

效能平台化实战

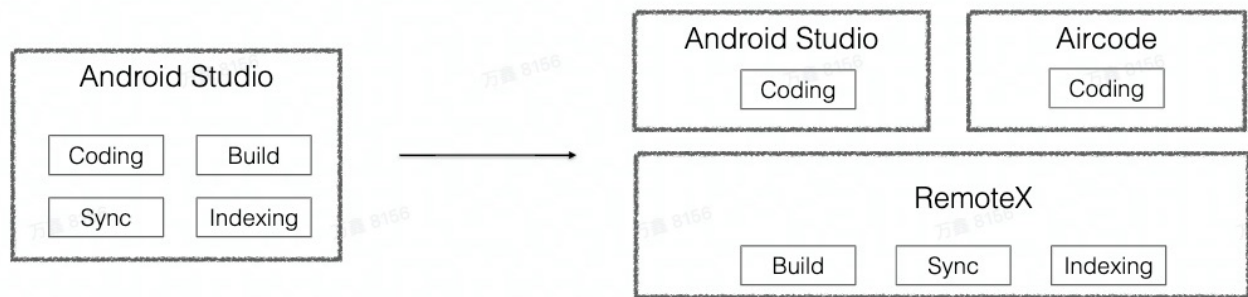
构建速度

编译越来越慢
电脑顶不住了

研发机器资源存在上限
工程增长速度 > 换机周期

本地编译不可持续

云端化



- 内存：-10G
- 耗时：-28%

问题变多

表像

- 问题变多
- 日常OnCall变多
- 架构投入成为瓶颈

深层次

- 问题发现：沉默的大多数，如何触达？
- 问题分析：有效问题筛选变成难题
- 问题解决：解决十分依赖人工

思路

问题收集平台

主动：效能洞察系统

被动：问题反馈渠道

问题分析专家系统

问题聚合定位分析

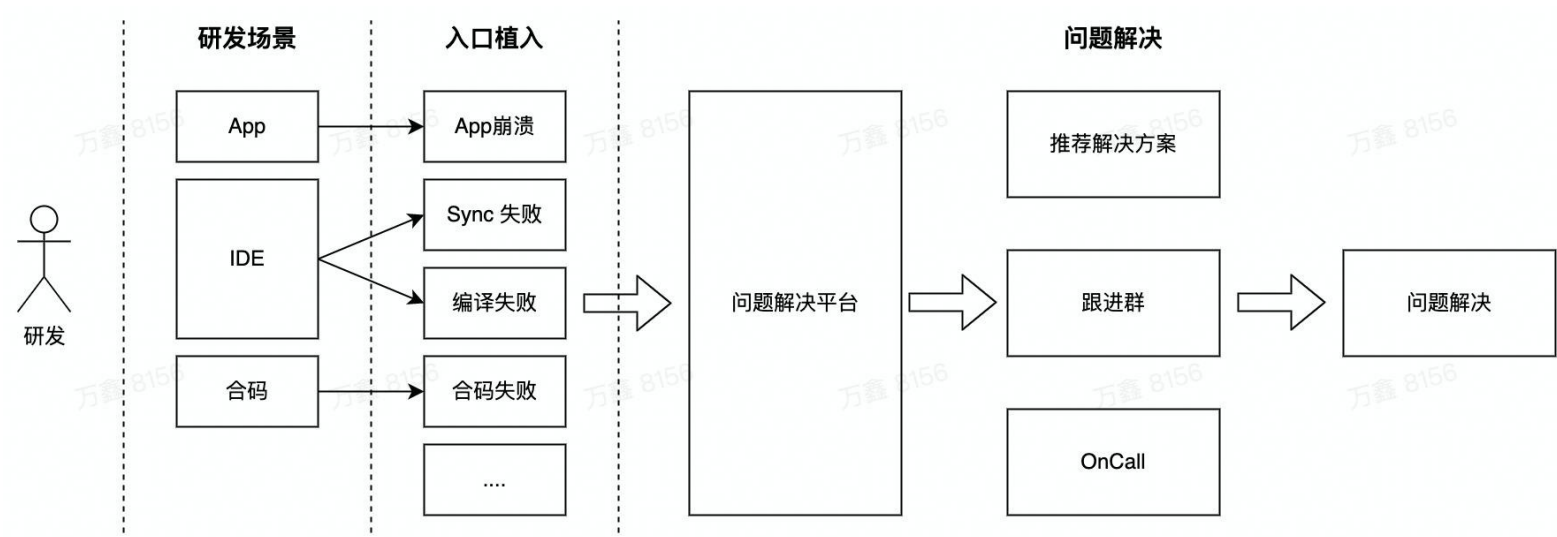
研发&工程数据中心

问题解决平台

解决问题分发

解决方案沉淀

问题解决平台



05

挑战与展望

我们遇到了什么问题？

代码

- 架构不统一
- 代码查阅不便
- 大型重构困难
- 冗余代码多

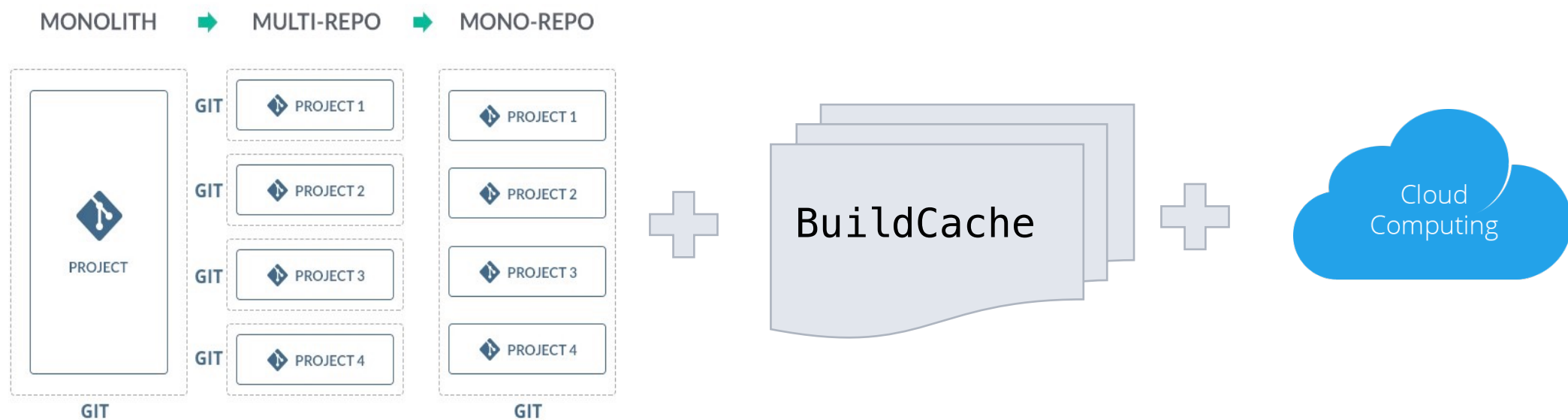
工具

- 多仓与分支管理
成本高，Bug多
- 兼容性差，问题多

流程

- 主干分支不稳定
- CI/CD 流程多，耗时久
- 回滚困难

如何应对

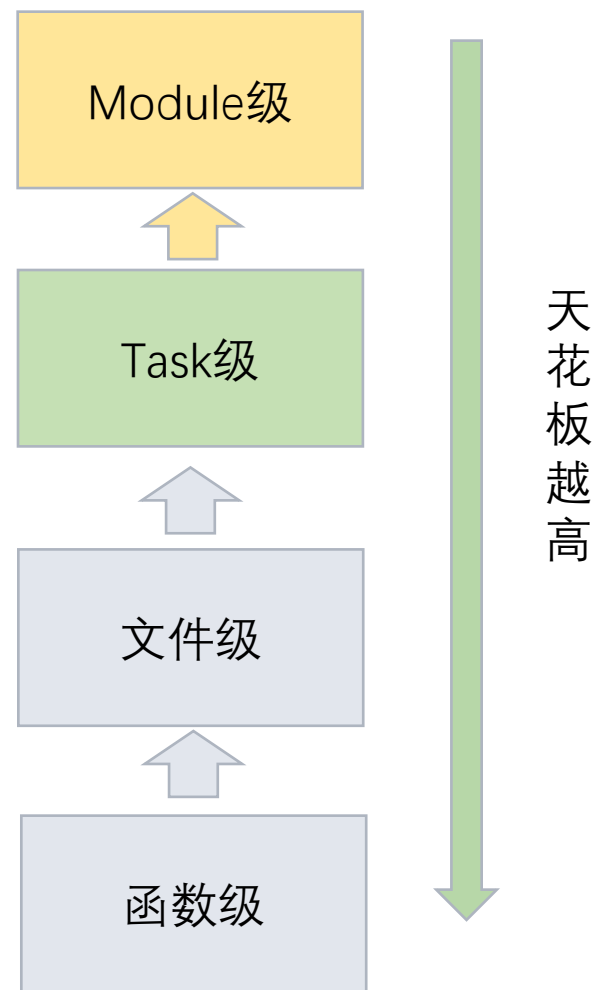
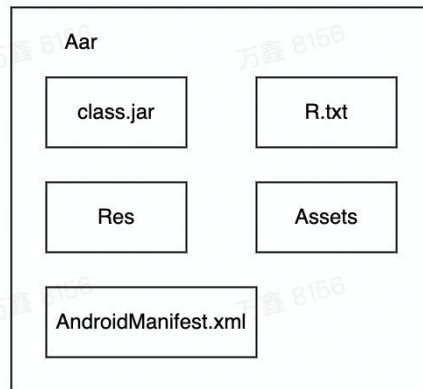
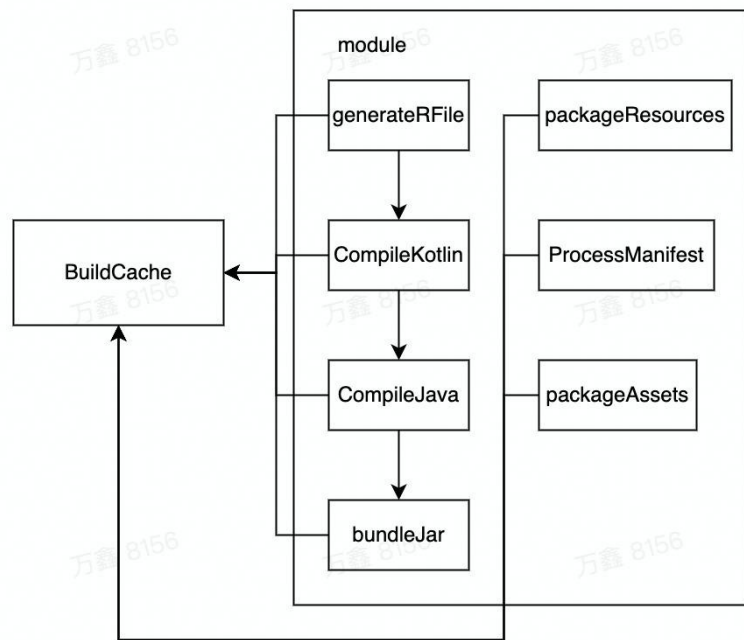


收益与挑战

- **兼容性问题**：彻底解决引入二进制之后的IDE、编译、运行兼容性问题，工程鲁棒性更高；
- **学习成本**：单仓模型简单，不涉及复杂的仓库和分支管理；更利于保持工程架构、规范一致性，降低学习成本；
- **协同成本**：代码对所有人透明，有利于更好合作和跨团队代码贡献；
- **代码集成**：取消组件发布耗时，加大合码吞吐率，封板效率高；

- **权限问题**：修改权限；Pull Request；等配套系统需要同步修改；
- **性能问题**：超大代码规模导致编译、IDE、Git等工具带来显著性能问题；

编译颗粒度优化





THANKS.